F215343: Rx: Enabler - Correct RX Access for Carved-out Members

NGX-F215343: Rx Enabler (Arch) – Correct RX Access for Carved-out Members

Feature Summary

Objective: Prevent display or use of pharmacy claims and prescriptions that fall outside a member's Aetna-covered period when the member's pharmacy history originates from CVS Caremark. CVS systems legitimately maintain a longitudinal (lifetime) prescription history for an individual, spanning time periods when the person may have had other insurers or no coverage at Aetna. When Aetna consumes CVS data via the Consolidated Claims and related APIs, those lifetime records can include fills that pre-date (and in some cases post-date) the member's Aetna plan effective dates. If unfiltered, these records would be visible in Aetna Health and could be acted upon by downstream services.

Solution: Introduce a Policy Effective Window Filter that gates visibility and use of any CVS-sourced prescription claim, order, or refill. The filter constrains records to the intersection of: (a) the member's pharmacy membership effective window that aligns with the Aetna plan effective dates, (b) the client's requested startDate/endDate, and (c) attestation-controlled integration state. Only claims with fillDate within this window are eligible for display and subsequent flows (detail, order, refill). The integration itself is governed by the 'CVSRxCarveOut' feature toggle/attestation; when enabled, Aetna presents embedded capabilities and SSO; when disabled, Aetna presents a limited, link-out experience.

- NGX-F215343: Rx Enabler (Arch) – Correct RX Access for Carved-out Members
- Feature Summary
- Discovery
- Options
  - Option 1: Centralized Eligibility & Display Logic Service
  - Option 2: Distributed Logic in Clients
- Solution Sketch
- API (OpenAPI)
- Service Logic
- Services
- Non-Functional Requirements (NFR)
- Observability (Metrics, Traces, Logs)
- Scenarios
    - Mode A Scenario — PLP list for Aetna-covered window (fetch-and-filter)
    - Mode B Scenario — ETL validates raw CVS claims (filter-only)
  - Partial-success (list/batch) scenarios:
- Client Changes
- Field Mapping Tables
- Example Payloads
- Scenarios
- API & Swagger (Pharmacy Service v1)
  - Acquisition & Filtering Modes (Explicit)

        - Mode A — Service fetch-and-filter (recommended for PLP/PDB, PlaceOrder, Ship Consent, i90)
        - Mode B — Filter-only (caller passes raw claims; used by new stories/ETL/review queues)
    - Overview
    - Versioning & Stability
    - Error Model & Failure IDs
    - Partial Results Semantics (200 vs 206)
    - Brief Examples
    - Artifacts
- Services (Expanded)
- Appendix: Swagger Summary (Pharmacy Service v1)
    - Artifact Hyperlinks
- Appendix A: Interface Contracts & Field Mappings
- Appendix B: Failure Catalog

Discovery

End-to-end prescription management touches multiple services and rules. The discovery synthesized the following dependencies, implications, and constraints that must be addressed uniformly to avoid inconsistent behavior:

1) Data & Authorization Dependencies:

- CVS Auth (patient profile) issues a token used by Aetna to call CVS backends.
- Consolidated Claims (/getClaimsHistory) returns all visible plan-member claims for the authenticated member.
- Core Proxy (/userauthorizations) and Member Profile (/familyaccessrules) supply authorization and dependent visibility.
- UAF privacy checks further restrict access (e.g., HIPAA privacy restrictions, sensitive medications).

2) Filtering & Normalization Dependencies:

- Policy Effective Window Filter uses pharmacy membership effectiveStart/effectiveEnd to bound results.
- Request window validation requires startDate and endDate; errors if missing.
- Deduplicate by uniqueRxId and retain the latest fillDate for list responses.
- Final restriction by membershipResourceIds if provided.

3) Integration Surface Area:

- Five key integrations (PLP, PDB, PlaceOrder, Ship Consent, i90) depend on consistent claim visibility.
- ~47 related APIs rely on the same underlying data and rules; front-end logic cannot diverge.
- A crosswalk/mapping layer is required to align identifiers and payloads across systems.

4) Behavioral Edge Cases to Handle:

- Claims with fillDate before plan effectiveStart: exclude.
- Orders/refills that straddle coverage boundaries: exclude or block action if outside window.
- RefillsLeft = 0 or missing: ensure accurate messaging and no spurious eligibility.
- Attestation off (no integrated carve-out): present limited, link-out experience.
- Missing dates or invalid ranges: return 400 with actionable error details.

5) Caching, Observability, and Security:

- Cache key includes account idSource~idValue, membershipResourceId, startDate, endDate.
- Log decision points (authorization, privacy, effective window trimming, dedup) for auditability.
- Enforce HIPAA and PHI handling standards; secure token exchange and storage.

Options

Option 1: Centralized Eligibility & Display Logic Service

Centralize claim visibility rules in a middleware service that evaluates attestation flags, plan effective dates, and carve-out status before returning data to clients. This ensures consistency across all integrations (PLP, PDB, PlaceOrder, Ship Consent, i90) and eliminates undocumented front-end logic.

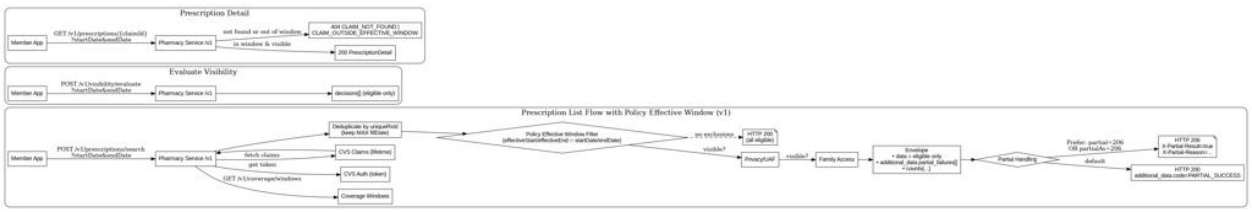
Option 2: Distributed Logic in Clients

Embed a shared validation library in each integration to enforce claim visibility rules locally. This offers independence but risks divergence over time.

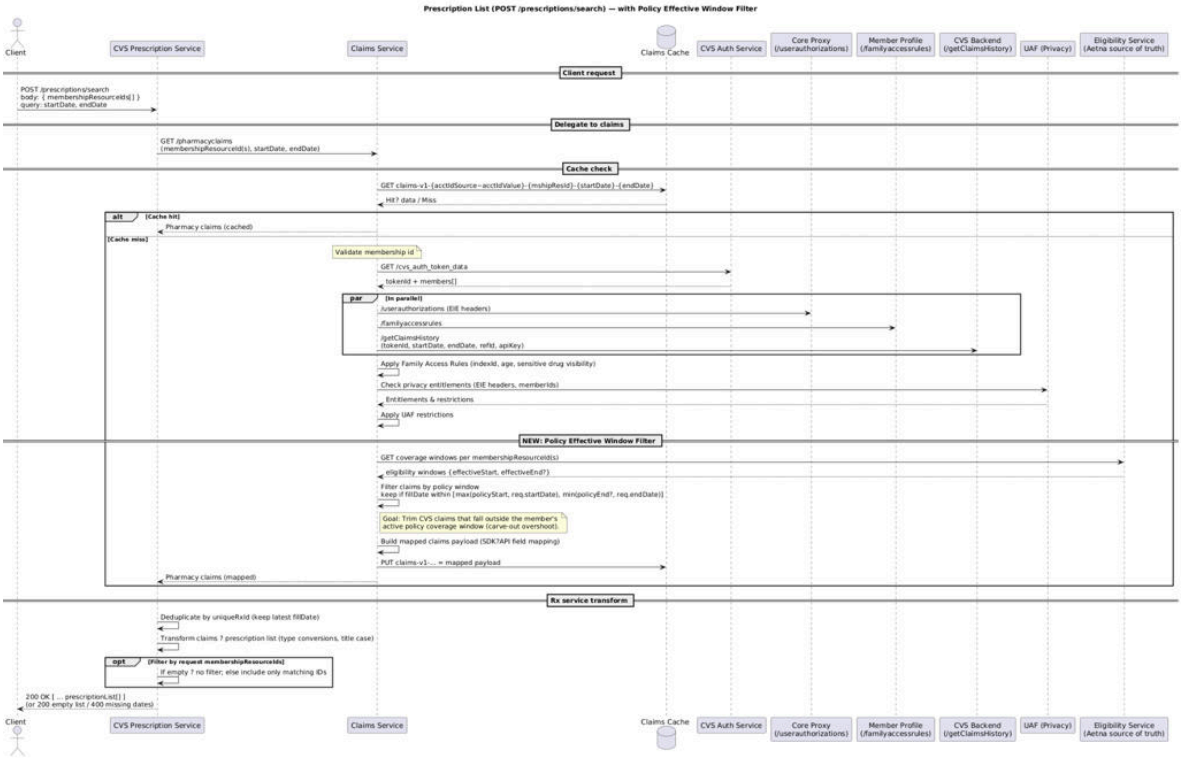
Selected: Option 1 – Centralized service approach for scalability and compliance.

Solution Sketch

Updated Diagram (aligned to /v1 & 200/206):



The following diagram illustrates the updated Prescription List Flow with Effective Date Policy Filtering applied. It replaces the previous orchestration placeholder and demonstrates where effective date filtering is enforced.



API (OpenAPI)

https://apiforge.cvshealth.com/apis/editor/Microservices/ah-prescription-effective-date-management/0.0.2

Service Logic

The centralized eligibility service enforces claim visibility rules in this order:

1. Validate attestation (CVSRxCarveOut) and integration state.
2. Retrieve CVS token (patient profile), authorizations, family access rules, and privacy entitlements.
3. Fetch claims and normalize; deduplicate by uniqueRxId keeping the latest fillDate.
4. Apply Policy Effective Window Filter using pharmacy membership effective dates ∩ request window.
5. Apply family access/UAF privacy restrictions.
6. Restrict by membershipResourceIds if provided and return mapped responses.

Services

- CVS Auth Service (patient profile token)
- CVS Consolidated Claims Service
- Prescription List Service (POST /prescriptions/search; GET /prescriptions/detail)
- Order & Refill Service
- Core Proxy (/userauthorizations)

- Member Profile (/familyaccessrules)
- UAF (privacy)
- Centralized Eligibility Middleware (Effective Date & Attestation Filtering)

Non-Functional Requirements (NFR)

SLOs (Pharmacy Service v1):

- Latency: P95 ≤ 300 ms (list), ≤ 250 ms (detail), ≤ 300 ms (evaluate).
- Availability: 99.9% monthly; error budget 43m/month.
- Partial rate baseline (list): ≤ 25% in steady state; investigate spikes.
- Timeout policy: upstream dependency timeouts at 3s; circuit-breaker after 5 consecutive timeouts.

Performance: Response ≤ 200ms under normal load.

Scalability: Support ~47 APIs and five key integrations.

Caching: Use deterministic cache keys and TTL aligned with CVS freshness.

Observability: Emit metrics and traces for cache hits, trims by window, privacy filters, and dedup.

Reliability: Graceful degradation when dependencies are unavailable.

Required metrics:

- requests_total{endpoint,method}
- trimmed_by_effective_window_total
- hidden_by_privacy_total, hidden_by_family_total
- deduplicated_total
- upstream_latency_ms{dependency} (histogram)
- end_to_end_latency_ms (histogram)
- Traces/Logs:
- CorrelationId (EIE transaction id) on every hop
- Decision logs at INFO with reasons and counts
- Error envelopes with additional_data.failure_id and diagnostics

Observability (Metrics, Traces, Logs)

Scenarios

Mode A Scenario — PLP list for Aetna-covered window (fetch-and-filter)

Client: Aetna Health PLP

Call: *POST /v1/prescriptions/search?startDate=2019-01-01&endDate=2019-12-31*

Body:

```
{
    "membershipResourceIds": ["5~263801696+31+1+20180101+788678+C+3"],
     "options": {"applyPrivacy": true, "applyFamilyAccess": true, "applyDeduplication": true, "requireAttestation": true}
}
```

Outcome: returns eligible prescriptions (deduped) for in-window fills; out-of-window claims are excluded and, if any exist, are listed in additional_data.partial_failures[] with failure_id=CLAIM_OUTSIDE_EFFECTIVE_WINDOW.

Mode B Scenario — ETL validates raw CVS claims (filter-only)

Client: nightly quality job

Call: *POST /v1/visibility/evaluate?startDate=2019-01-01&endDate=2019-12-31&partialAs=206*

Body:

```
{
    "memberContext": {"membershipResourceIds": ["5~263801696+31+1+20180101+788678+C+3"], "attestation": {"cvsrxCarveOut": true}},
    "claims": [
      {"claimNumber": "200023611694001", "uniqueRxId": "9751485501674528227", "fillDate": "2019-08-12", "membershipResourceId": "5~..."},
      {"claimNumber": "200023611694002", "uniqueRxId": "9751485501674528228", "fillDate": "2018-04-12", "membershipResourceId": "5~..."}
    ],
    "options": {"applyPrivacy": true, "applyFamilyAccess": true, "applyDeduplication": true}
 }
```

Outcome: service returns 206 with X-Partial-Result=true. decisions[] includes the in-window claim as eligible; the out-of-window claim is captured in additional_data.partial_failures[].

Partial-success (list/batch) scenarios:

- Mixed eligibility: return HTTP 200 with additional_data.code=PARTIAL_SUCCESS by default.
- Client may request HTTP 206 via Prefer: partial=206 or partialAs=206.
- Eligible items appear in data; excluded items are listed in additional_data.partial_failures[].
- UI: show eligible prescriptions; display a non-blocking info banner: “Some items were hidden due to coverage or privacy.”

1. Prescription claim before coverage effectiveStart → excluded from all views and actions.
2. Refill/order outside coverage window → blocked with clear messaging.
3. Member without signed attestation → link-out (no embedded capabilities).
4. Member with signed attestation → full embedded integration.

5. Missing startDate/endDate → 400 Bad Request with guidance.
6. Sensitive drug + privacy restriction → excluded per UAF.

Client Changes

- Integrate with centralized eligibility service for consistent filtering.
- Enforce and display messaging when items are hidden due to effective dates or privacy.
- Respect attestation state to show embedded features vs link-out.
- Keep UI and mobile parity across platforms.

Field Mapping Tables

Prescription List (POST /prescriptions/search):

- .details[].memberClaims[].claimNumber → prescriptionList[].claimId
- .details[].memberClaims[].uniqueRxId → prescriptionList[].prescriptionId
- .details[].memberClaims[].drug.name → prescriptionList[].drugName (Title Case)
- .details[].memberClaims[].drug.strength → prescriptionList[].drugStrength (String→Number)
- .details[].memberClaims[].daysSupplyQuantity → prescriptionList[].daysSupply (String→Number)
- .details[].memberClaims[].dispensedQuantity → prescriptionList[].quantity (String→Number)
- .details[].memberFirstName → memberFirstName
- .details[].memberLastName → memberLastName
- .details[].memberClaims[].fillDate → prescriptionList[].lastFilledDate (Dedup: keep latest fillDate)
- .details[].memberClaims[].prescriberName → prescriptionList[].prescriberName
- .details[].memberClaims[].pharmacyName → prescriptionList[].fulfilledBy
- .details[].relationshipToSubscriber → relationshipToSubscriber
- .details[].membershipResourceId → membershipResourceId
- prescriptionsRequest.startDate → startDate
- prescriptionsRequest.endDate → endDate

Prescription Detail (GET /prescriptions?claimId=...):

- .details[].memberClaims[].claimNumber → claimId

- .details[].memberClaims[].uniqueRxId → prescriptionId
- .details[].memberClaims[].fillDate → lastFilledDate
- .details[].memberFirstName → memberFirstName
- .details[].memberLastName → memberLastName
- .details[].relationshipToSubscriber → relationshipToSubscriber
- .details[].membershipResourceId → membershipResourceId
- .details[].memberClaims[].drug.name → drugName
- .details[].memberClaims[].drug.strength → drugStrength
- .details[].memberClaims[].drug.dosageForm → drugForm
- .details[].memberClaims[].daysSupplyQuantity → daysSupply
- .details[].memberClaims[].dispensedQuantity → quantity
- .details[].memberClaims[].refillsLeft → refillsLeft
- .details[].memberClaims[].prescriber.fullName → prescriberName
- .details[].memberClaims[].pharmacyName → fulfilledBy
- (.memberClaims[].payAmount + .clientPayAmount) → estimatedCost
- .details[].memberClaims[].rxNumber → prescriptionNumber
- .details[].memberClaims[].drug.ndcId → NDC11

Example Payloads

Prescription List Response (truncated example):


```
[
  {
    "membershipResourceId": "5~263801696+31+1+20180101+788678+C+3",
    "memberFirstName": "John",
    "memberLastName": "Doe",
    "relationshipToSubscriber": "Self",
    "startDate": "2019-04-01",
    "endDate": "2019-12-05",
    "prescriptionList": [
      {
        "claimId": "200023611694001",
        "prescriptionId": "674529584",
        "drugName": "Lipitor",
        "drugStrength": "200mg",
        "quantity": 15,
```

```
        "lastFilledDate": "2018-04-12",
        "daysSupply": 5,
        "fulfilledBy": "Caremark prescription service",
        "prescriberName": "John Doe"
      }
    ]
  }
]
```

Prescription Detail Response (truncated example):

```
{
  "claimId": "200023611694001",
  "prescriptionId": "674529584",
  "drugName": "Lipitor",
  "drugStrength": "200mg",
  "drugForm": "Tablet",
  "daysSupply": 5,
  "quantity": 15,
  "lastFilledDate": "2018-04-12",
  "refillsLeft": 0,
  "prescriberName": "John Doe",
  "fulfilledBy": "Caremark prescription service",
  "estimatedCost": 111,
  "prescriptionExpirationDate": "2018-04-12",
  "prescriptionNumber": "674529584",
  "NDC11": "536012297"
}
```

Scenarios

1. Prescription claim with fillDate before coverage effectiveStart → excluded from both list and detail.
2. Refill order straddling effective window (e.g., refill date before effectiveStart, claim date within) → excluded.
3. Refill request with refillsLeft = 0 → disallowed with clear message.
4. Member without attestation (CVSRxCarveOut = false) → UI shows link-out, not embedded services.
5. Missing startDate/endDate query parameters → service throws 400 Bad Request.
6. Claims flagged as sensitive (isSensitive=true) → excluded if UAF rules deny visibility.
7. Dependent under 18 without family access rules → excluded unless explicitly authorized.
8. Duplicate claims with same uniqueRxId → keep only claim with latest fillDate.

API & Swagger (Pharmacy Service v1)

Acquisition & Filtering Modes (Explicit)

The Pharmacy Service supports two integration modes, both enforcing the same Policy Effective Window, attestation, UAF privacy, family access rules, and deduplication by uniqueRxId. Choose the mode based on where claims are acquired.

Mode A — Service fetch-and-filter (recommended for PLP/PDB, PlaceOrder, Ship Consent, i90)

• Call POST /v1/prescriptions/search (and GET /v1/prescriptions/{claimId} for detail).

• Service obtains CVS Auth (patient profile token), calls Consolidated Claims (/getClaimsHistory), then normalizes, deduplicates (uniqueRxId, keep latest fillDate), and applies the Policy Effective Window = [pharmacy membership effectiveStart/effectiveEnd] ∩ [request startDate/endDate].

• UAF privacy and family access are applied; attestation (CVSRxCarveOut) gates embedded capabilities.

• Partial results: HTTP 200 with additional_data.code=PARTIAL_SUCCESS by default, or opt-in HTTP 206 via Prefer: partial=206 or partialAs=206.

Mode B — Filter-only (caller passes raw claims; used by new stories/ETL/review queues)

• Call POST /v1/visibility/evaluate with claims[].

• Service treats inputs as the source of truth for items to evaluate and applies the same policy stack: effective window, UAF privacy, family access, and dedup.

• Returns decisions[] plus a normalized PrescriptionSummary for visible items.

• Partial results semantics identical to Mode A (200 default, or 206 with headers on opt-in).

Overview

This section defines the authoritative API contract for the Pharmacy Service enforcing the Policy Effective Window Filter across list, detail, and generic visibility use cases. All routes are prefixed with /v1. The full OpenAPI 3.0.3 specification is attached separately; this section provides a concise summary and examples.

Endpoints (v1):

• GET /v1/health – service probe

• GET /v1/coverage/windows – resolve Aetna-aligned effective coverage windows for one or more members

• POST /v1/prescriptions/search – authoritative list (window trim, privacy/family access, dedup)

• GET /v1/prescriptions/{claimId} – authoritative detail view

• POST /v1/visibility/evaluate – generic batch evaluator for new/adjacent use cases

Versioning & Stability

All endpoints are versioned under /v1. Breaking changes will result in a new major prefix (/v2). Non-breaking, additive fields are gated via tolerant readers and do not change the version path.

Error Model & Failure IDs

Responses are wrapped in a standard envelope. For errors (4xx/5xx), the API returns an ErrorEnvelope with additional_data carrying canonical failure identifiers and reasons. These are suitable for telemetry and client-side branching.

Common failure_id values:

- MISSING_REQUIRED_DATE – startDate and/or endDate missing
- INVALID_DATE_FORMAT – dates not ISO YYYY-MM-DD
- INVALID_DATE_RANGE – startDate after endDate
- NOT_ATTESTED – embedded capabilities disallowed by attestation
- CLAIM_NOT_FOUND – claimId not associated to visible member
- CLAIM_OUTSIDE_EFFECTIVE_WINDOW – claim exists but fillDate outside effective window
- HIDDEN_BY_PRIVACY / HIDDEN_BY_FAMILY_ACCESS – filtered by UAF/family rules
- INVALID_MEMBERSHIP_ID – malformed membershipResourceId
- TOKEN_EXPIRED / UPSTREAM_UNAVAILABLE – dependency failures

Partial Results Semantics (200 vs 206)

When a list/batch contains a mix of eligible and ineligible items, the API does not fail the request. By default it returns HTTP 200 with additional_data.code=PARTIAL_SUCCESS, placing only eligible items in data and capturing excluded items under additional_data.partial_failures[].

Clients can opt into explicit partial semantics via either:

- Header: Prefer: partial=206
- Query: partialAs=206

When used, the service responds with HTTP 206 and includes headers X-Partial-Result=true and X-Partial-Reason.

Brief Examples

POST /v1/prescriptions/search?startDate=2019-01-01&endDate=2019-12-31

Request body (subset):

```
{
  "membershipResourceIds": [
    "5~263801696+31+1+20180101+788678+C+3"
  ],
  "options": {
    "applyPrivacy": true,
    "applyFamilyAccess": true,
    "applyDeduplication": true,
    "requireAttestation": true
```

```
  }
}
```

Response 200 (partial success, truncated):

```
{
    "statusCode": 200,
    "statusDescription": "OK",
    "additional_data": {
      "code": "PARTIAL_SUCCESS",
      "partial": true,
      "counts": {"total": 2, "eligible": 1, "hiddenByEffectiveWindow": 1},
      "partial_failures": [{
        "failure_id": "CLAIM_OUTSIDE_EFFECTIVE_WINDOW",
        "diagnostics": {"claimId": "200023611694002", "fillDate": "2018-04-12"}
      }]
    },
    "data": [{
      "membershipResourceId": "5~263801696+31+1+20180101+788678+C+3",
      "prescriptionList": [{"claimId": "200023611694001", "drugName": "Lipitor", "lastFilledDate":
"2019-08-12"}]
    }]
}
```

POST /v1/visibility/evaluate?startDate=2019-01-01&endDate=2019-12-31&partialAs=206 (client opts into 206)

Response 206 (partial content, truncated):

```
{
    "statusCode": 200,
    "statusDescription": "OK",
    "additional_data": {"code": "PARTIAL_SUCCESS", "partial": true},
    "data": {"decisions": [{"claimNumber": "200023611694001", "visibility": "eligible"}]}
}
```

Headers: X-Partial-Result: true; X-Partial-Reason: CLAIM_OUTSIDE_EFFECTIVE_WINDOW

Artifacts

- OpenAPI 3.0.3: pharmacy-service-v1d.yaml
- Postman: pharmacy-service.postman_collection.v2.json, pharmacy-service.postman_environment.json
- Client stubs: pharmacy-service-client-stubs.ts

Services (Expanded)

Below are the core services and their interactions, expanded with field-level mappings and sample payloads to ensure precise architectural alignment.

• CVS Auth Service (`GET /cvs_auth_token_data`): issues token with member demographics and identifiers.

• Consolidated Claims Service (`GET /pharmacyclaims`): retrieves raw claim history across visible plan members.

• Prescription List Service (`POST /prescriptions/search`): applies filtering, deduplication, effective date window.

• Prescription Detail Service (`GET /prescriptions?claimId=...`): enriches a single prescription with dosage, refills, cost.

• Order & Refill Service: manages refill eligibility and order placement with effective date enforcement.

• Family Access Rules (`/familyaccessrules`): ensures dependent entitlements are respected.

• UAF Service: privacy restrictions enforced per member and medication.

• Centralized Eligibility Middleware: authoritative Policy Effective Window enforcement.

Appendix: Swagger Summary (Pharmacy Service v1)

openapi: 3.0.3

info:

  title: Pharmacy Service

  version: 1.0.0

servers:

  - url: https://sandbox.aetnahealth.example.com

paths:

  /v1/prescriptions/search: POST

  /v1/prescriptions/{claimId}: GET

  /v1/visibility/evaluate: POST

  /v1/coverage/windows: GET

x-partial-handling:

```
  default_http_status_for_partials: 200
  override:
    header: "Prefer: partial=206|200"
    query: "partialAs=206|200"
ErrorEnvelope:
  statusCode: integer
  statusDescription: string
  additional_data:
    failure_id: string
    failure_reason: string
    diagnostics: object
    code: "PARTIAL_SUCCESS"?
    partial: boolean?
    counts: object?
    partial_failures: [ { failure_id, failure_reason, diagnostics } ]
  data: any | null
Example — POST /v1/prescriptions/search?startDate=2019-01-01&endDate=2019-12-31
Request:
{
  "membershipResourceIds": ["5~263801696+31+1+20180101+788678+C+3"],
  "options": {"applyPrivacy": true, "applyFamilyAccess": true, "applyDeduplication": true,
"requireAttestation": true}
}
Response (200, PARTIAL_SUCCESS):
{
  "statusCode": 200,
  "statusDescription": "OK",
  "additional_data": {
```

"code": "PARTIAL_SUCCESS", "partial": true,

"counts": {"total": 2, "eligible": 1, "hiddenByEffectiveWindow": 1}

},

"data": [{ "membershipResourceId": "5~...",

"prescriptionList": [{ "claimId": "200023611694001", "drugName": "Lipitor", "lastFilledDate": "2019-08-12" }] }]

}

Example — POST /v1/visibility/evaluate?startDate=2019-01-01&endDate=2019-12-31&partialAs=206

Response (206, headers: X-Partial-Result=true; X-Partial-Reason=CLAIM_OUTSIDE_EFFECTIVE_WINDOW):

{

"statusCode": 200,

"statusDescription": "OK",

"additional_data": { "code": "PARTIAL_SUCCESS", "partial": true },

"data": { "decisions": [{ "claimNumber": "200023611694001", "visibility": "eligible" }] }

}

Artifact Hyperlinks

OpenAPI YAML: pharmacy-service-v1d.yaml

Postman Collection: pharmacy-service.postman_collection.v2.json

Postman Environment: pharmacy-service.postman_environment.json

TypeScript Client Stubs: pharmacy-service-client-stubs.ts

Appendix A: Interface Contracts & Field Mappings

CVS Source Field	Aetna Field	Transform/Type	Notes
details[].memberClaims[].claimNu	prescriptionList[].claimId	string → string	Primary identifier for claim detail

mber			
details[].memberClaims[].uniqueRxId	prescriptionList[].prescriptionId	string → string	Dedup key; choose latest fillDate
details[].memberClaims[].drug.name	prescriptionList[].drugName	string → string (Title Case)	
details[].memberClaims[].drug.strength	prescriptionList[].drugStrength	string → string	
details[].memberClaims[].dispensedQuantity	prescriptionList[].quantity	string → integer	
details[].memberClaims[].daysSupplyQuantity	prescriptionList[].daysSupply	string → integer	
details[].memberClaims[].fillDate	prescriptionList[].lastFilledDate	date → date	Windowing & dedup
details[].memberClaims[].pharmacyName	prescriptionList[].fulfilledBy	string → string	
details[].memberClaims[].prescriber.fullName	prescriptionList[].prescriberName	string → string	
details[].memberFirstName / memberLastName	memberFirstName / memberLastName	string → string	
details[].relationshipToSubscriber	relationshipToSubscriber	string → string	
details[].membershipResourceId	membershipResourceId	string → string	Scoping & coverage lookup

memberClaims[].drug.ndcId	NDC11 (detail)	string → string	Detail view only

Appendix B: Failure Catalog

failure_id	Reason	HTTP Status	Remediation / Client Guidance
MISSING_REQUIRED_DATE	startDate and/or endDate missing	400	Provide both dates in ISO format
INVALID_DATE_FORMAT	Date format not ISO YYYY-MM-DD	400	Fix formatting; retry
INVALID_DATE_RANGE	startDate after endDate	400	Correct dates
NOT_ATTESTED	Attestation required for embedded capabilities	403	Use link-out or enable attestation
CLAIM_NOT_FOUND	No claim with provided id for caller	404	Verify claimId and membership context
CLAIM_OUTSIDE_EFFECTIVE_WINDOW	Claim exists but outside coverage window	404 (detail) / partial in list	Adjust dates; excluded in list/batch
HIDDEN_BY_PRIVACY	UAF/privacy restrictions	403 (evaluate) / partial in list	Surface banner; follow policy
HIDDEN_BY_FAMILY_ACCESS	Family access rules deny visibility	403 (evaluate) / partial in list	Change permissions

INVALID_MEMBERSHIP_ID	Malformed membershipResourceId	400	Correct identifier
TOKEN_EXPIRED	Upstream token invalid/expired	401/500	Refresh token; retry
UPSTREAM_UNAVAILABLE	Dependency timeout/error	500	Retry with backoff; alert on sustained failures